TASK 3

Understanding the Assumptions of Classification Algorithms

Machine Learning Lab

A classifier is a supervised machine learning algorithm that learns from labelled examples and predicts a category or class for new observations. Different classifiers make different assumptions about data structure, feature relationships, or class boundaries. Understanding these assumptions helps us select, prepare, and evaluate a model.

1. Logistic Regression

How it works: Logistic Regression estimates the probability that an observation belongs to a class. In binary classification, it applies the sigmoid function to a weighted combination of input features and predicts a class using a threshold.

Main assumptions:

- The log-odds of the target class have a linear relationship with the predictors. This does not mean the probability itself must be linear in each feature.
- Observations are independent. Repeated or grouped data may need a method that accounts for this structure.
- Predictors do not have severe multicollinearity, since highly correlated features can make coefficient estimates unstable.
- There are enough examples from each class to estimate the model reliably.
- There are no highly influential outliers that dominate the fitted coefficients.

Practical understanding: Normality of the input features and equal feature variances between classes are not required. Logistic Regression can still predict if assumptions are imperfect, but coefficient interpretation and probability estimates may be affected.

2. K-Nearest Neighbors (KNN)

How it works: KNN predicts the class of a new observation by finding its K closest training examples and using their labels, commonly through majority voting.

Main assumptions:

- Nearby observations tend to have similar class labels, so distance in feature space is informative.
- The selected distance metric is meaningful for the data.
- Features are on comparable scales, or are scaled before distance calculations, so large-valued features do not dominate.
- The training data contain enough representative examples near the observations to be classified.
- There are not too many irrelevant or redundant dimensions that make distances uninformative.

Practical understanding: KNN does not assume a specific probability distribution or a linear decision boundary. Its performance depends on K, the distance metric, scaling, data density, and feature quality.

3. Decision Tree

How it works: A Decision Tree repeatedly divides data using feature-based rules. Each split aims to form groups that are more consistent in their target class, and a terminal leaf produces the prediction.

Main assumptions:

- Useful class patterns can be represented through a sequence of feature-based splits.
- Training observations are representative of the cases on which the tree will be used.
- Tree complexity is controlled, because a very deep tree can memorize noise and overfit.
- Missing values and categorical features are handled appropriately by the selected implementation.

Practical understanding: Decision Trees do not require linearity, normality, or feature scaling. Their key practical concern is overfitting and instability, so depth limits, minimum leaf sizes, pruning, and validation are useful.

4. Random Forest

How it works: Random Forest combines many decision trees. Each tree is trained with randomized data and feature selection, and the trees' predictions are combined, commonly by majority vote.

Main assumptions:

- The training data are representative of the population where predictions will be made.
- Features contain information that helps distinguish the target classes.
- Randomization creates some diversity among trees; averaging works best when trees do not all make identical errors.
- The number and complexity of trees are sufficient to learn patterns without unnecessary computation.
- Data leakage is avoided: predictors do not include information unavailable at prediction time.

Practical understanding: Random Forest does not require normally distributed features, linearity, or feature scaling. It is generally less sensitive than a single tree to small data changes, but can still be affected by biased data, leakage, and uninformative features.

5. Support Vector Machine (SVM)

How it works: SVM finds a decision boundary that separates classes while maximizing the margin around it. Kernel functions can represent non-linear boundaries in the original feature space.

Main assumptions:

- A useful separating boundary exists in the original or selected kernel feature space. A soft-margin SVM can tolerate some overlap and errors.
- The selected kernel and its parameters represent the data structure reasonably well.
- Numerical features are usually scaled because distances and dot products are central to SVM.
- Regularization is selected to balance a wide margin against training errors.
- Training observations are representative, since points near the margin can strongly influence the boundary.

Practical understanding: SVM does not require normally distributed features. A kernel can model non-linear boundaries, but performance depends on scaling, kernel choice, and parameter tuning.

6. Naive Bayes

How it works: Naive Bayes applies Bayes' theorem to estimate class probabilities from feature values. Its 'naive' assumption is that features are conditionally independent once the class is known.

Main assumptions:

- Features are conditionally independent given the target class. This is often an approximation rather than a perfect description of real data.
- The likelihood distribution matches the feature type: Gaussian Naive Bayes is intended for continuous features modelled with class-conditional Gaussian distributions, while Multinomial Naive Bayes is often used for counts.
- Training data include examples of the classes and feature patterns needed to estimate priors and likelihoods.
- Training class proportions and feature patterns are reasonably representative if reliable probability estimates are required.

Practical understanding: The central assumption is conditional independence, not overall independence between features. Distributional assumptions depend on the Naive Bayes variant. The method can still classify effectively when independence is only approximate.

Summary Comparison

Classifier	Central assumption / requirement	Usually needs scaling?
Logistic Regression	Linear relationship between predictors and target log-odds; independent observations; no severe multicollinearity.	Not strictly; often useful
KNN	Nearby points tend to share labels; distance is meaningful.	Yes, usually
Decision Tree	Useful patterns can be captured by feature splits; complexity must be controlled.	No
Random Forest	Trees learn useful patterns and have enough diversity for ensemble averaging.	No
SVM	A useful separating boundary exists in the chosen feature/kernel space.	Yes, usually
Naive Bayes	Features are conditionally independent given the class; likelihood model suits feature type.	Usually no

Conclusion

No classifier is completely assumption-free. Logistic Regression and Naive Bayes make explicit statistical assumptions, while KNN, tree-based methods, and SVM depend more on distance, data structure, or the existence of a useful boundary. Assumptions guide model selection and diagnostics; they do not guarantee success. In practice, classifiers should be evaluated on unseen data using suitable measures such as precision, recall, F1-score, accuracy, and a confusion matrix.